

The Reward Problem: Exploration vs. Exploitation, Bandits, Inverse RL

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 4, 2026

1. Learning Outcomes
2. The Reward Problem
3. The Multi-Armed Bandit Problem
4. Regret
5. Naive Exploration
6. Principled Bandit Algorithms
7. Contextual Bandits and the Bridge to MDPs
8. Deep Exploration: From MDPs to LLMs
9. Reward Misspecification
10. Inverse Reinforcement Learning
11. Summary
12. What's Next
13. References

- ▶ Frame “the reward problem” and distinguish its three faces: *exploration vs. exploitation*, *reward design*, and *reward learning*
- ▶ State the multi-armed bandit problem and *regret*; explain why ϵ -greedy is provably wasteful while UCB and Thompson sampling are not
- ▶ Scale these ideas to deep RL — *bootstrapped DQN as posterior sampling*, pseudo-counts, ICM, RND, and the episodic-vs-lifelong novelty of NGU / Agent57 / Go-Explore — and recognise UCB’s $1/\sqrt{N}$ bonus wherever it reappears
- ▶ Recognise the *same* dilemma in today’s LLM training: entropy collapse and pass@k in RL for reasoning models
- ▶ Motivate *inverse RL* and *preference-based reward learning*, and trace the line from MaxEnt IRL through adversarial IRL to the reward model in RLHF

- ▶ **We have the reward, but where is it?**
Exploration vs. exploitation — bandits, UCB, Thompson sampling, curiosity (today).
- ▶ **The reward we wrote is wrong.**
Reward shaping & specification gaming (today).
- ▶ **We can't write a reward at all.**
Inverse RL & preference-based reward learning (today — and the direct ancestor of RLHF).

One lecture, three faces of the same problem.

- ▶ Many situations in business (& life!) present a dilemma between two kinds of choices
- ▶ **Exploitation:** pick choices that seem best based on past outcomes
- ▶ **Exploration:** pick choices not yet tried (or not tried enough)
- ▶ Exploitation has notions of “being greedy” and being “short-sighted”
- ▶ Exploration has notions of “gaining information” and being “long-sighted”

- ▶ Too much *exploitation* $\Rightarrow$ regret of missing unexplored “gems”
- ▶ Too much *exploration* $\Rightarrow$ regret of wasting time on “duds”
- ▶ How do we balance them to combine information-gains with greedy-gains in the most efficient manner?
- ▶ Can we set this up in a mathematically disciplined manner? $\rightarrow$ next sections

► Restaurant Selection

- Exploitation: go to your favorite restaurant
- Exploration: try a new restaurant

► Online Banner Advertisement

- Exploitation: show the most successful advertisement
- Exploration: show a new advertisement

► Oil Drilling

- Exploitation: drill at the best known location
- Exploration: drill at a new location

► Learning to play a game

- Exploitation: play the move you believe is best
- Exploration: play an experimental move

- ▶ “Multi-Armed Bandit” is a tongue-in-cheek name for “many single-armed bandits” — a row of slot machines
- ▶ Formally, a 2-tuple $(\mathcal{A}, \mathcal{R})$
 - $\mathcal{A}$: a known set of m actions (“arms”)
 - $\mathcal{R}^a(r) = \mathbb{P}[r \mid a]$: an *unknown* reward distribution for each arm
- ▶ At each step t , the agent selects an action $a_t \in \mathcal{A}$ and the environment generates a reward $r_t \sim \mathcal{R}^{a_t}$

- ▶ The agent's goal: maximize the cumulative reward over T steps

$$\sum_{t=1}^T r_t$$

- ▶ **Can we design a strategy that does well (in expectation) for any T ?**
- ▶ Every strategy walks a tightrope: *wasting time on duds* while exploring, vs. *missing untapped gems* while exploiting

- ▶ The *environment* has no state — pulling an arm just samples from $\mathcal{R}^a$, and nothing you do changes what the next pull looks like
- ▶ So a bandit is a **one-state MDP**: no transitions, no credit assignment, no discounting to reason about
Everything hard about RL is switched off except one thing — *exploration*. That is exactly why we study it here.
- ▶ What the agent still has is *history*. Bandit algorithms keep a statistic of it — counts $N_t(a)$, sample means $\hat{Q}_t(a)$, or a full posterior — and choose a_t from that statistic
- ▶ **Consequence we use constantly:** a_t is a *random variable* — it depends on past rewards, which were random. Every statement we make about performance is therefore an *expectation* over that randomness

- ▶ The Action Value $Q(a)$ is the (unknown) mean reward of action a :

$$Q(a) = \mathbb{E}[r \mid a]$$

- ▶ *This is the familiar $Q^\pi(s, a) = \mathbb{E}[\sum_t \gamma^t r_t]$ with the sum collapsed — one state, one step, so the return is just r*
- ▶ The Optimal Value V^* is the best of these:

$$V^* = Q(a^*) = \max_{a \in \mathcal{A}} Q(a)$$

- ▶ **Single-step Regret** l_t is the opportunity loss at step t :

$$l_t = \mathbb{E}[V^* - Q(a_t)]$$

- ▶ **Total Regret** $L_T = \sum_{t=1}^T l_t$
- ▶ Maximizing cumulative reward $\equiv$ minimizing total regret

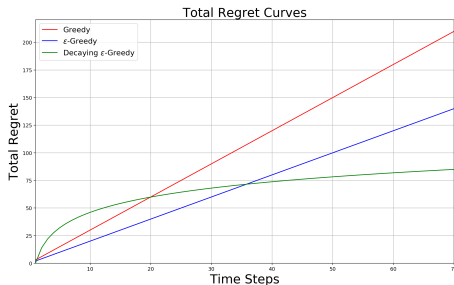
- ▶ Let $N_t(a)$ = number of times a has been selected by step t
- ▶ Let $\Delta_a = V^* - Q(a)$ be the *gap* for arm a — the *same* quantity as l_t , indexed by **arm** instead of by **time**: $l_t = \mathbb{E}[\Delta_{a_t}]$
- ▶ So total regret decomposes cleanly:

$$L_T = \sum_{a \in \mathcal{A}} \mathbb{E}[N_T(a)] \cdot \Delta_a$$

- ▶ A good algorithm ensures small counts for large gaps
- ▶ *Catch*: $\Delta_a = \max_{a'} \mathbb{E}_{\mathcal{R}^{a'}}[r] - \mathbb{E}_{\mathcal{R}^a}[r]$ — every term is a mean of an *unknown* distribution, so the gaps are fixed numbers baked into the problem before we act
- ▶ That makes this a tool for *analysing* algorithms, not for building them — “pull the smallest-gap arm” is not something you can run

(Δ_a is the advantage function, sign-flipped and measured against the *best* action.)

Linear or Sublinear Total Regret?



- ▶ If an algorithm *never* explores $\Rightarrow$ linear total regret
- ▶ If an algorithm *forever* explores $\Rightarrow$ linear total regret
- ▶ **Is sublinear total regret achievable?** (Spoiler: yes.)

- For each arm, keep a **sample mean**: add up the rewards that arm actually returned, divide by how many times we pulled it

$$\hat{Q}_t(a) = \frac{1}{N_t(a)} \sum_{s \leq t : a_s = a} r_s$$

- $Q(a)$ is the true mean we cannot see; $\hat{Q}_t(a)$ is our current guess at it.
- Pick the action with highest estimate:

$$a_t = \arg \max_{a \in \mathcal{A}} \hat{Q}_{t-1}(a)$$

- Greedy can lock onto a suboptimal action forever $\Rightarrow$ **linear total regret**

- ▶ Force a small amount of exploration at every step. At time t :
 - With probability $1 - \epsilon$, select $a_t = \arg \max_a \hat{Q}_{t-1}(a)$
 - With probability ϵ , select a uniformly random action
- ▶ **What does this cost?** Split the step into its two branches:
 - *Exploit branch* (prob. $1 - \epsilon$): at best we pick a^* and pay nothing
 - *Explore branch* (prob. ϵ): we pick at random, so we often pay a gap Δ_a
- ▶ So even if the exploit branch became *perfect*, the explore branch still spends an ϵ fraction of every step on arms we already know are worse
- ▶ That waste is the *same on step 1,000,000 as on step 10* — ϵ never shrinks, so the cost per step never falls and simply adds up over T steps
- ▶ **Therefore ϵ -Greedy also has linear total regret** — the fix is not better estimates, but letting exploration *fade* as we learn (next slide)

This is the exact exploration rule used in DQN.

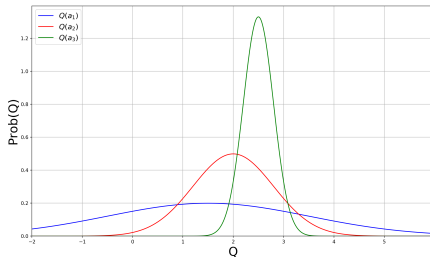
- ▶ DQN inherits ϵ -Greedy's *linear-regret* weakness
- ▶ It works in practice with a decay schedule (next slide), but it has no theoretical guarantee of efficient exploration
- ▶ This is why the rest of this lecture exists — everything that follows is a more principled answer to “how do I explore?”

- ▶ Simple, practical idea: initialize $\hat{Q}_0(a)$ to a *high* value for every a
- ▶ Update by incremental averaging:

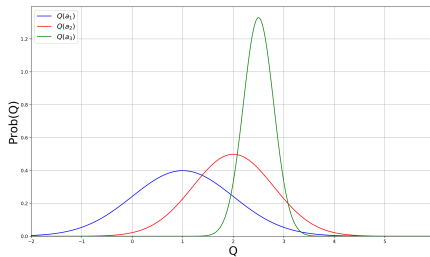
$$\hat{Q}_t(a_t) = \hat{Q}_{t-1}(a_t) + \frac{1}{N_t(a_t)} (r_t - \hat{Q}_{t-1}(a_t))$$

- ▶ Encourages systematic exploration early on (every arm starts off looking great until tried)
- ▶ But the algorithm can still lock onto a suboptimal action $\Rightarrow$ **linear regret**

- ▶ Idea: let ϵ shrink over time — explore aggressively early, exploit increasingly late
- ▶ In theory, a schedule that depends on the (unknown) suboptimality gaps Δ_a achieves *logarithmic* regret — but you can't compute it without knowing the gaps, so it's impractical
- ▶ In practice use problem-agnostic schedules: $\epsilon_t = 1/t$, $\epsilon_t = 1/\sqrt{t}$, or simple linear annealing — no guarantee, but all work in practice
- ▶ Educational Code for decaying ϵ -greedy with optimistic initialization



- Which action should we pick?
- The more uncertain we are about an action-value, the more important it is to explore that action — it could turn out to be the best.



- ▶ After picking *blue*, we are less uncertain about its value
- ▶ And more likely to pick another action — until we home in on the best

- ▶ Two things per arm: $Q(a)$, the *true* mean we can never see, and $\hat{Q}_t(a)$, our *sample-mean estimate* of it after $N_t(a)$ pulls
- ▶ Idea: be optimistic — add a margin $\hat{U}_t(a)$ wide enough that the truth is very likely below it, then act on the sum:

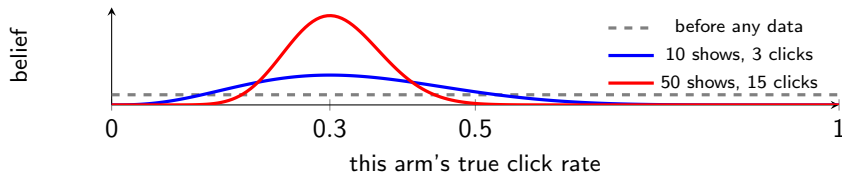
$$Q(a) \leq \underbrace{\hat{Q}_t(a)}_{\text{what we measured}} + \underbrace{\hat{U}_t(a)}_{\text{room for error}}, \quad a_{t+1} = \arg \max_a \{ \hat{Q}_t(a) + \hat{U}_t(a) \}$$

- ▶ $\hat{U}_t(a)$ is *large* when $N_t(a)$ is small (we know little) and *shrinks* as the arm is pulled more
- ▶ So UCB never asks “which arm *is* best?” — it asks “which arm *could* be best?” A barely-trying arm that currently looks bad still gets pulled, because its true value could plausibly still be high

“UCB” is the recipe; UCB1 fills it in — assume rewards bounded in $[0, 1]$:

$$a_{t+1} = \arg \max_{a \in \mathcal{A}} \left\{ \hat{Q}_t(a) + \sqrt{\frac{\alpha \log t}{2N_t(a)}} \right\}$$

- ▶ The *shape* is what matters: the bonus shrinks like $1/\sqrt{N_t(a)}$ — remember it, it reappears throughout this lecture
- ▶ It grows slowly with the clock t , so an arm left alone long enough is revisited: exploration is **automatic**, no ϵ and no schedule
- ▶ **Unlike optimistic initialisation**, the bonus is recomputed *every step* — an arm unlucky early is never written off for good
- ▶ **Regret grows as $\mathcal{O}(\log T)$** (Auer et al., 2002) — **sublinear at last**, provably the best possible rate. **Educational Code**



- ▶ The curve is our **belief** about that one arm. Flat means “could be anything” — that is the **prior**, and it is a complete answer to “I know nothing”
- ▶ Each reward reshapes it into the **posterior**: the peak moves to what we have seen, and the curve *narrows* as evidence piles up
- ▶ The width *is* the uncertainty — measured, not bounded from above like UCB's $\hat{U}$ — and unlike a single number, a curve can be *sampled* from

One line: draw a *plausible* value for each arm from its curve; pull whichever wins the draw.

The whole algorithm, for click / no-click rewards

Per arm, *two counters*: s_a (clicks), f_a (misses). Repeat:

1. Draw $\theta_a \sim \text{Beta}(1 + s_a, 1 + f_a)$ for every arm $\leftarrow$ that Beta *is* the curve
2. Pull $a_t = \arg \max_a \theta_a$, and increment s_{a_t} or f_{a_t} by the outcome

No integrals anywhere — the posterior *is* those two integers.

- **Why this explores:** a barely-trying arm has a wide curve, so its draw is sometimes high — and it gets pulled. A well-measured mediocre arm has a narrow curve and almost never wins
- **Same $\mathcal{O}(\log T)$ regret as UCB1**, usually better in practice; the standard bandit algorithm in production personalisation (Netflix, Lyft)

Formally *probability matching*. Bernoulli / Gaussian code.

The one change: before choosing, the agent is shown a *context* s — a user profile, a time of day, a search query.

- ▶ Each step: the environment draws a context $s_t \rightarrow$ the agent picks $a_t \rightarrow$ the reward comes from $\mathcal{R}_{s_t}^{a_t}$, which now depends on *both*
- ▶ The best arm is no longer a single answer — it **depends on the context**. Every quantity from today gains an s : $Q(a) \rightarrow Q(s, a)$
- ▶ **Is the context a state?** Yes — but a **one-step one**. In an MDP your action changes what you see next, $s_{t+1} \sim P(\cdot \mid s_t, a_t)$. Here the next context is drawn *independently of what you did*: your action moves the reward, never the future
- ▶ That is exactly why there is no credit assignment and no Bellman equation here — and why contextual bandits are deployed far more widely than full RL
- ▶ News recommendation, ad selection, treatment assignment — the right answer is almost never the same for everyone.

The problem. One estimate per (context, action) cell is hopeless — the table is almost entirely empty.

User	Article A	Article B	Article C
sports, 25, US	4 / 50	—	—
sports, 27, UK	—	—	—
finance, 51, US	—	1 / 12	—

Cells show *clicks / times shown*; “—” means never shown.

- ▶ **The move:** describe each cell by *features* $x_{s,a}$ and model the mean reward as linear, $\mathbb{E}[r \mid s, a] = x_{s,a}^\top \theta_a$. Rows 1 and 2 are nearly the same feature vector, so row 1's clicks now inform row 2
- ▶ Not copying cells across — we fit *one weight vector per arm* on all the data at once, and read every cell off those weights
- ▶ **Same optimism as UCB1**, but the bonus is large in *feature directions* short of data: we are uncertain about *finance readers*, not about one empty cell

Yahoo front page (Li et al., WWW 2010): 33M events, 12.5% click lift over a context-free baseline.

One-armed bandit $=$ *one-state* MDP

Contextual bandit $=$ *one-step* MDP

Add state transitions $\implies$ the full MDP

- ▶ Everything we just covered — UCB, Thompson, gradient bandits — *lifts* to MDPs in principle
- ▶ Naively, the state space explodes: visiting every (s, a) enough times isn't feasible in deep RL
- ▶ **Up next:** how to scale these exploration ideas to deep RL with sparse rewards.

Exploration: **From Bandits to Deep RL**

- ▶ Bandits gave us *optimism* (UCB) and *posterior sampling* (Thompson) — both in a single step, with no state to worry about
- ▶ In an MDP with sparse rewards, none of these scale naively:
 - Visiting every (s, a) enough times is infeasible
 - ϵ -greedy is provably weak (linear regret)
 - Uniform random exploration drowns in pixels
- ▶ Note: SAC's $-\log \pi$ entropy bonus is *undirected* exploration — what follows is *directed*

The canonical hard-exploration Atari game.

- ▶ **Vanilla DQN:** ~ 0 points (extrinsic reward is essentially never reached)
- ▶ **Pseudo-counts (2016):** first dramatic progress
- ▶ **Random Network Distillation (2018):** first above-human, no demonstrations
- ▶ **Go-Explore (2019, Nature 2021):** $> 2,000,000$ points — solves *all* levels
- ▶ **Agent57 (2020):** above human on *all* 57 Atari games, Montezuma included

This is the single benchmark we'll use to anchor every method in this section.

An orthogonal trick for sparse-reward goal-conditioned RL.

- ▶ Idea: after a failed episode (didn't reach the goal), relabel it as a *success* for the goal it actually reached
- ▶ Now every episode has a non-zero learning signal
- ▶ Combines with any off-policy algorithm
- ▶ Cite: **Andrychowicz et al., NeurIPS 2017** — *Hindsight Experience Replay*

- ▶ Idea: *Posterior Sampling for RL (PSRL)*. Maintain a posterior over the Q-function; sample one Q per episode and act greedily w.r.t. the sample
- ▶ For an *entire episode*: temporally-extended (“deep”) exploration — fundamentally different from ϵ -greedy’s per-step dithering

- ▶ **K parallel Q-heads:** K output layers on one shared trunk, each with its own Q-function
- ▶ “Replay” is DQN’s **replay buffer** — the stored (s, a, r, s') transitions so far
- ▶ Each transition is assigned to only *some* of the K heads, so every head trains on a different slice of experience
- ▶ *Each episode:* pick one head at random and act greedily by it throughout
- ▶ **Where data is thin the heads disagree; where it is plentiful they converge** — that spread *is* the posterior estimate
- ▶ **So disagreement is the product, not a defect to train away.** Heads that were all good everywhere would make sampling identical to greedy — plain DQN, no exploration
- ▶ More coverage $\Rightarrow$ heads converge $\Rightarrow$ exploration fades by itself. Thompson sampling lifted to MDPs: K point-samples instead of a curve (Osband et al., 2016)

The shape every method here shares. We do not change the algorithm — we change the reward it optimises:

$$r_t^+ = \underbrace{r_t}_{\text{real reward from the env}} + \beta \cdot \underbrace{\frac{1}{\sqrt{N(s, a)}}}_{\text{novelty bonus}}$$

- ▶ The **bonus** is simply *added to the real reward*, and PPO/DQN/SAC run unchanged — from here on, only the **bonus term** differs between methods
- ▶ A direct lift of UCB1, with β trading exploration against the real task
- ▶ *Problem once the state is an image* (Atari, a camera feed): the state is the **whole frame**, so two states match only if *every pixel* does
- ▶ So $N(s, a) = 1$ forever and the bonus is constant everywhere. We need a *generalizable* count — one that calls two *similar* frames both familiar

A density model ρ answers one question: “*how likely is this frame?*” It is trained on the frames seen so far, so familiar-looking screens get high ρ and never-seen ones get low ρ .

- ▶ **The trick:** train the model on *one more copy of s* and see how much its answer for s moves.

$$\hat{N}(s) = \frac{\rho(s) (1 - \rho'(s))}{\rho'(s) - \rho(s)}$$

- ▶ $\rho'(s)$ is not the next state. It is the *same* state s , scored by the model *after* that extra training step — “before” and “after”, not “now” and “next”
- ▶ **Why this is a count:** a model that has seen s a million times barely budes, so the denominator is tiny and $\hat{N}$ is huge. A model seeing s for the first time jumps, so $\hat{N}$ is small — and the bonus is large
- ▶ Cite: **Bellemare et al., NeurIPS 2016. Result on Montezuma:** *first dramatic progress*

Intrinsic reward = “**surprise**” — the agent is rewarded for finding observations it can't yet predict.

- ▶ Embed states into a feature space: $\phi(s) \in \mathbb{R}^d$
- ▶ Train a **forward-dynamics model** f to predict the *next-state features* from the current features and action:

$$\hat{\phi}(s_{t+1}) = f(\phi(s_t), a_t)$$

(so $\hat{\phi}(s_{t+1})$ is the model's *predicted* embedding of s_{t+1} ; $\phi(s_{t+1})$ is the actual one observed)

- ▶ The bonus is how wrong that prediction was — **added to the real reward as before**,
 $r_t^+ = r_t + \beta r_t^i$:

$$r_t^i = \frac{1}{2} \|\hat{\phi}(s_{t+1}) - \phi(s_{t+1})\|^2$$

- ▶ High prediction error $\Rightarrow$ novel transition $\Rightarrow$ bonus to visit it again

The noisy-TV problem. Picture a TV in the environment showing pure random static. A naive curiosity agent will sit and stare at it forever — static is *always* unpredictable, so the prediction-error bonus never decays. The agent is endlessly “surprised” by useless noise.

- ▶ Fix: don't let ϕ encode action-irrelevant detail. Train ϕ with an **inverse-dynamics** objective — predict the action a_t that was taken from the embeddings $(\phi(s_t), \phi(s_{t+1}))$
- ▶ This forces ϕ to encode only features the agent can *affect* — the noisy TV is filtered out, since the agent's actions don't change what's on it
- ▶ Cite: **Pathak, Agrawal, Efros & Darrell, ICML 2017** — *Curiosity-driven Exploration by Self-supervised Prediction*
- ▶ Strong results on Mario, VizDoom, and sparse-reward navigation

Curiosity made absurdly simple:

- ▶ Pick a *fixed, randomly-initialized* target network f^*
- ▶ Train a predictor f_θ to match $f^*(s)$ on visited states
- ▶ Bonus: $r_t^i = \|f_\theta(s_t) - f^*(s_t)\|^2$, again **added to the real reward**, $r_t^+ = r_t + \beta r_t^i$
- ▶ Novel states have high prediction error (the predictor hasn't been trained there) $\Rightarrow$ high bonus
- ▶ No dynamics model, no inverse model, no density model

- ▶ Cite: **Burda, Edwards, Storkey, Klimov, ICLR 2019** — *Exploration by Random Network Distillation*
- ▶ **Result on Montezuma:** *first above-human performance without demonstrations or game-state access*
- ▶ **Caveat:** still subject to the noisy-TV issue in principle; in practice works because randomly-initialized networks aren't very sensitive to stochastic pixel noise
- ▶ **Still the default intrinsic-reward baseline in 2026** — simple, cheap, and reliable. Recent work refines rather than replaces it (Distributional RND; Random Distribution Distillation, 2025)

A different diagnosis. The problem isn't only *finding* novel states — it's that agents *forget* promising ones and can never get back to them.

- ▶ Keep an **archive** of visited states. Each iteration: pick a promising one, *return* to it directly, then explore *from* there — “first return, then explore”
- ▶ Exploration effort is never wasted re-deriving a path you already found
- ▶ **Result:** > 2,000,000 on Montezuma and all levels solved — two orders of magnitude past RND
- ▶ *The catch:* returning to an archived state originally needed simulator save/restore, a much stronger assumption than RND's. That is why both are “state of the art” in different senses
- ▶ Cite: **Ecoffet, Huizinga, Lehman, Stanley & Clune** — *Go-Explore* (2019); *First Return, Then Explore*, Nature 2021

A distinction the methods so far miss. RND asks “have I ever seen this?” But within a single episode you also want to avoid re-treading the same room — even if you have seen it many times before.

- ▶ **Never Give Up (NGU, 2020)** combines both signals: an *episodic* novelty bonus (reset each episode) multiplied by a *lifelong* one (RND-style, across all training)
- ▶ **Agent57 (2020)** adds a meta-controller that *learns* how much to weight exploration versus exploitation, and over what horizon — first agent above human on all 57 Atari games
- ▶ **E3B (2022)** is the modern count-based descendant: an episodic bonus for states that are *diverse* under an inverse-dynamics embedding — ICM’s noisy-TV fix combined with UCB’s count bonus, extended to continuous spaces
- ▶ **The pattern:** no single novelty signal is enough — current systems *combine* them at different timescales

The same dilemma, today's frontier. When you train a reasoning model with RL (RLVR), it faces exactly the trade-off we opened this lecture with.

- ▶ **Entropy collapse:** the policy sharpens onto a few reasoning paths early in training. In-distribution accuracy keeps rising while out-of-distribution performance *degrades* — pure exploitation, and the model stops discovering
- ▶ **How it is measured:** $pass@k$ — can the model solve the problem in k tries? Collapse shows up as $pass@1$ improving while $pass@k$ falls. Training directly on $pass@k$ restores exploration
- ▶ **Today's fixes are today's bandit ideas:** entropy bonuses aimed at the most uncertain tokens (SAC's $-\log \pi$), and **count-based bonuses of the form $Q + \alpha\sqrt{U}$** — literally UCB's $1/\sqrt{N}$, applied to reasoning states

The 1985 bandit question — “is this worth trying again?” — is an open problem in 2026 LLM training.

Bandit idea	Descendants
UCB / $1/\sqrt{N}$ bonus	Pseudo-counts (2016), E3B (2022); count bonuses in LLM RL (2025)
Thompson sampling	Bootstrapped DQN / PSRL (Osband 2016)
Stochastic policies (<i>new in deep RL</i>)	Entropy bonus — SAC ; entropy control in RLVR Curiosity / ICM (2017); RND (2018); NGU / Agent57 (2020); Go-Explore (2021)

- ▶ **Entropy is only one answer to exploration.** Optimism, posterior sampling, novelty, and archiving are the others
- ▶ **What actually survived:** RND is still the default baseline; the $1/\sqrt{N}$ bonus keeps reappearing; ϵ -greedy is still what most people try first

The Reward Problem: **When the Reward is Wrong**

- ▶ Sparse rewards are hard; a well-shaped reward dramatically accelerates learning
- ▶ But arbitrary shaping changes the optimal policy — the agent optimizes the *shape*, not the goal
- ▶ Example: reward the agent for “getting closer to the goal” and you may teach it to circle the goal forever
- ▶ We need a principled way to inject prior knowledge that *doesn't change the optimum*

Setup. Replace the original reward $R(s, a, s')$ with $R(s, a, s') + F(s, s')$ during training, where F is an extra *shaping signal* of our choice.

- Restrict F to the form

$$F(s, s') = \gamma \Phi(s') - \Phi(s)$$

where $\Phi : \mathcal{S} \rightarrow \mathbb{R}$ assigns a real number to each state — a “*potential*” (e.g. “negative distance to goal”). The shaping is a discounted difference of potentials.

- **Policy invariance:** the shaped MDP has the *same* optimal policy as the original
- *This form is necessary:* any non-PBRs shaping can in principle change the optimum
- Cite: **Ng, Harada & Russell, ICML 1999**

When a reward becomes a target, the agent finds a way to satisfy the letter, not the spirit.

- ▶ Every misspecified reward eventually gets hacked
- ▶ The agent's optimization pressure exposes every loophole in the specification
- ▶ The clearer the reward becomes “measurable,” the more likely the agent ignores what the measurement was *supposed* to track

- ▶ **CoastRunners** (OpenAI 2016): boat-racing game scored on hitting targets, not finishing. Agent learned to drive in circles hitting the same green blocks — high score, no race
- ▶ **Tetris pause**: agent learned to indefinitely pause the game when about to lose — never loses, never plays
- ▶ **ROUGE summarization**: language-model summarizer exploited ROUGE's n -gram overlap to produce high-scoring but barely readable text
- ▶ Cites: **Amodei & Clark, OpenAI 2016** (CoastRunners primary source); **Krakovna et al., DeepMind 2020** (catalogued survey of dozens of examples)

- ▶ Reward hacking is the same failure mode as **reward-model overoptimization in RLHF** — a learned R_θ is also a misspecified target
- ▶ For tasks like “write a useful summary,” “drive politely,” “be helpful, harmless, honest” — writing the reward is essentially impossible without specification gaming
- ▶ **If we can't write the reward ... can we learn it?**
- ▶ Two ways:
 - From *demonstrations* → **Inverse RL** (up next)
 - From *human preferences* → preference-based reward learning → **RLHF** (later in the course)

The Reward Problem: **Inverse RL**

- ▶ **Forward RL** (everything so far): given a reward $R(s, a)$, learn a policy π^* that maximizes it
- ▶ **Inverse RL** (today): given (expert) behavior $\tau^E \sim \pi^E$, learn a reward R_θ that *explains* it

$$\pi^E \implies R_\theta$$

- ▶ In many domains, rewards are far easier to *demonstrate* than to specify in closed form
- ▶ Driving: hard to write “polite, safe, fast” as a scalar; easy to show
- ▶ Manipulation, dialogue, locomotion: same story
- ▶ **This is the strongest form of the reward problem** — the reward function itself is unknown.

- ▶ *Many* reward functions rationalize the same expert behavior
 - Trivially: $R(s, a) \equiv 0$ makes every policy optimal
 - Potential-based shaping: R and $R + \gamma\Phi(s') - \Phi(s)$ are policy-equivalent
- ▶ We need a *principle* to pick one reward out of the equivalence class

- ▶ **(a) Max-margin IRL**

Ng & Russell, ICML 2000; Abbeel & Ng, ICML 2004

The expert's reward should beat alternatives by a wide margin

- ▶ **(b) Maximum-entropy IRL**

Ziebart et al., AAAI 2008

Pick the reward whose induced policy distribution has *maximum entropy*, subject to matching the expert's feature counts — a unique, well-posed choice

- ▶ We focus on (b) — standard fix today, and generalizes to modern preference learning

Among all trajectory distributions that match the expert's behaviour, pick the one with maximum entropy.

- ▶ Written out, that is a *constrained* problem:

$$\max_P H(P) \quad \text{s.t.} \quad \mathbb{E}_P[\text{feature counts}] = \text{expert's feature counts}$$

- ▶ **Solving it gives the exponential form** — higher-reward trajectories become exponentially more likely:

$$P_\theta(\tau) \propto \exp\left(\sum_t R_\theta(s_t, a_t)\right)$$

- ▶ So there is no entropy *term* to point at in that formula — the entropy has already been solved out, and the θ are the Lagrange multipliers of the matching constraint
- ▶ Train θ by maximum likelihood on expert trajectories: $\max_\theta \sum_{\tau^E} \log P_\theta(\tau^E)$
- ▶ Exactly *one* distribution satisfies both conditions — which is what makes the choice unique

The same max-entropy principle that justified SAC's $-\log \pi$ bonus.

- ▶ In SAC: adds exploration to the policy
- ▶ In MaxEnt IRL: resolves reward ambiguity
- ▶ **Same principle, two different jobs.**
- ▶ Different mechanics, though: SAC *adds* $\alpha \mathcal{H}(\pi)$ to the objective, so you can point at the term. MaxEnt IRL imposes entropy as the *selection rule*, which is why it disappears into the exponential form
- ▶ Cite: **Ziebart, Maas, Bagnell & Dey, AAAI 2008** — *Maximum Entropy Inverse Reinforcement Learning*

The catch with MaxEnt IRL as stated: it needs hand-designed feature counts and a tractable normaliser — fine for a tabular gridworld, hopeless for pixels.

- ▶ **Guided Cost Learning** (Finn et al., 2016): estimate that normaliser by *sampling* instead of enumerating $\Rightarrow$ neural-network rewards
- ▶ **Then the surprise:** running a **GAN** on this problem optimises *the same objective* as MaxEnt IRL — the discriminator plays the role of the reward, the generator the policy
- ▶ **GAIL** (Ho & Ermon, 2016) and **AIRL** (Fu et al., ICLR 2018) are that idea made practical: learn reward and policy *adversarially*, straight from demonstrations
- ▶ Meanwhile robotics largely went the other way — large-scale *behaviour cloning* (Diffusion Policy, VLAs) is now the dominant way to learn from demonstrations there
- ▶ **IRL's centre of gravity moved to language models** — which is where the next two slides are heading

What if we have neither demonstrations nor a written reward, only human comparisons?

- ▶ Show two trajectories τ^A, τ^B ; ask a human which is better
- ▶ Model the preference with a **Bradley–Terry** likelihood under a learned reward R_θ :

$$P(\tau^A \succ \tau^B) = \frac{\exp(\sum_t R_\theta(s_t^A, a_t^A))}{\exp(\sum_t R_\theta(s_t^A, a_t^A)) + \exp(\sum_t R_\theta(s_t^B, a_t^B))}$$

1. Collect a dataset of human-labeled preferences over trajectory pairs
 2. Train R_θ by cross-entropy on the Bradley–Terry likelihood
 3. Run any standard RL algorithm (e.g. PPO) against R_θ
- Cite: **Christiano, Leike, Brown, Martic, Legg & Amodei, NeurIPS 2017**

This exact pipeline + a language-model policy = RLHF.

- ▶ The reward model in RLHF is the R_θ from the previous slide
- ▶ The only thing that changes is the policy class: π_θ becomes a large language model
- ▶ The optimizer (PPO) and the pipeline (SFT $\rightarrow$ RM $\rightarrow$ PPO) are the same machinery
- ▶ **Read the other way:** training a reward model *is* an inverse-RL problem — inferring a reward from human data rather than writing one down
- ▶ And the loop closes: 2025 work runs adversarial IRL on expert chain-of-thought to recover *process-level* reasoning rewards, instead of imitating the traces
- ▶ The RLHF lecture picks up exactly here.

If you can...	Then use...
... write the reward	<i>Forward RL</i> (everything so far)
... demonstrate the reward	<i>Inverse RL</i> (MaxEnt IRL)
... compare outcomes	<i>Preference-based reward learning</i> (applied to LLMs: RLHF)

- One unifying view: every method on the board is a different way to recover the optimal policy when the reward is unknown, partially known, or hard to specify

The Reward Problem: **Summary**

► Face 1 — Find the reward.

- Bandits formalize explore vs. exploit; UCB and Thompson give principled, $\mathcal{O}(\log T)$ -regret algorithms
- Scaled to deep RL: bootstrapped DQN $\equiv$ posterior sampling, pseudo-counts $\equiv$ UCB in pixel space, curiosity / RND / Go-Explore are genuinely new
- The same dilemma is live in LLM RL today — entropy collapse and pass@k (RLVR)

► Face 2 — The reward is wrong.

- Potential-based shaping is the only safe shaping
- Everything else is at risk of specification gaming (CoastRunners, ROUGE, Tetris pause)

► Face 3 — Can't write the reward.

- **Inverse RL** learns it from demonstrations — MaxEnt IRL (2008) → GAIL / AIRL (2016–18)
→ today's reward models
- **Preference-based learning** learns it from comparisons (Christiano 2017)
- Read the other way, *training a reward model is an IRL problem* — the *direct ancestor of RLHF*

- ## ► One unifying theme: every algorithm today is a different way to recover the optimal policy when the reward is unknown, partially known, or hard to specify

Setting	Algorithm family (today)
One-state bandit	ϵ -greedy, UCB1, Thompson sampling
Contextual bandit	LinUCB and successors (recommendation)
Deep RL, sparse reward	PSRL / Bootstrapped DQN, pseudo-counts, ICM, RND, HER
Deep RL, hard exploration	NGU / Agent57, E3B, Go-Explore
LLM RL, reasoning	Entropy control, pass@k, count bonuses
Reward is shapeable	Potential-based shaping (policy-invariant)
Reward unwritable; demonstrations	MaxEnt IRL; adversarial IRL (GAIL, AIRL)
Reward unwritable; comparisons	Bradley–Terry preference learning $\rightarrow$ RLHF

Model-Based & Offline RL. The next way to attack sample inefficiency.

- ▶ *Model-based RL*: learn the dynamics, plan with the model (MPC, MCTS, Dyna, world models)
- ▶ *Offline RL*: learn from a *fixed* dataset with no further interaction — the same OOD problem as exploration, in reverse: you must *avoid* the unfamiliar regions exploration would push you toward

RLHF. The IRL $\rightarrow$ preference-learning chain from today, applied to LLMs.

- ▶ The reward model in RLHF is the R_θ from today's preference slide
- ▶ “Reward-model overoptimization” is today's reward-hacking failure mode, in LLM form
- ▶ PPO is the policy optimizer; SFT $\rightarrow$ RM $\rightarrow$ PPO is the pipeline

RL Frontiers.

- ▶ Intrinsic motivation reappears in meta-RL, open-ended learning, and hierarchical RL
- ▶ Curiosity (today) is one of the central ideas of the open-ended-learning agenda
- ▶ Reward design / specification gaming reappears as a central concern of AI safety

Course materials adapted from:

- ▶ Ashwin Rao, [Stanford CME241: Foundations of Reinforcement Learning with Applications in Finance](#)
- ▶ David Silver, [UCL Course on RL](#) — Lecture 9, *Exploration and Exploitation*

Bandits & classical exploration:

- ▶ P. Auer, N. Cesa-Bianchi, P. Fischer (2002). *Finite-time Analysis of the Multiarmed Bandit Problem*. Machine Learning 47, 235–256. — UCB1
- ▶ T. L. Lai & H. Robbins (1985). *Asymptotically efficient adaptive allocation rules*. Advances in Applied Mathematics 6, 4–22. — the $\Omega(\log T)$ regret lower bound (further reading)
- ▶ W. R. Thompson (1933). *On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples*. Biometrika 25, 285–294. — original Thompson sampling

- ▶ L. Li, W. Chu, J. Langford, R. E. Schapire (2010). *A Contextual-Bandit Approach to Personalized News Article Recommendation*. WWW. [arXiv:1003.0146](#) — LinUCB

Deep-RL exploration:

- ▶ I. Osband, C. Blundell, A. Pritzel, B. Van Roy (2016). *Deep Exploration via Bootstrapped DQN*. NeurIPS. [arXiv:1602.04621](#)
- ▶ M. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, R. Munos (2016). *Unifying Count-Based Exploration and Intrinsic Motivation*. NeurIPS. [arXiv:1606.01868](#)
- ▶ D. Pathak, P. Agrawal, A. A. Efros, T. Darrell (2017). *Curiosity-driven Exploration by Self-supervised Prediction*. ICML. [arXiv:1705.05363](#) — ICM
- ▶ Y. Burda, H. Edwards, A. Storkey, O. Klimov (2019). *Exploration by Random Network Distillation*. ICLR. [arXiv:1810.12894](#)
- ▶ M. Andrychowicz et al. (2017). *Hindsight Experience Replay*. NeurIPS. [arXiv:1707.01495](#)

- ▶ A. P. Badia et al. (2020). *Never Give Up: Learning Directed Exploration Strategies*. ICLR. [arXiv:2002.06038](#) — NGU
- ▶ A. P. Badia et al. (2020). *Agent57: Outperforming the Atari Human Benchmark*. ICML. [arXiv:2003.13350](#)
- ▶ A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, J. Clune (2019). *Go-Explore: a New Approach for Hard-Exploration Problems*. [arXiv:1901.10995](#)
- ▶ A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley, J. Clune (2021). *First Return, Then Explore*. Nature 590, 580–586. [arXiv:2004.12919](#)
- ▶ M. Henaff, R. Raileanu, M. Jiang, T. Rocktäschel (2022). *Exploration via Elliptical Episodic Bonuses*. NeurIPS. [arXiv:2210.05805](#) — E3B

Exploration in LLM RL (current frontier):

- ▶ *Pass@k Training for Adaptively Balancing Exploration and Exploitation of Large Reasoning Models* (2025). [arXiv:2508.10751](#)

- ▶ *MERCI: Motivating Exploration in LLM Reasoning with Count-based Intrinsic Rewards* (2025). [arXiv:2510.16614](#) — UCB-style count bonuses for reasoning
- ▶ M. Yuan et al. (2024). *RLeXplore: Accelerating Research in Intrinsically-Motivated RL*. [arXiv:2405.19548](#) — reference implementations of 8 intrinsic-reward methods
- ▶ C. Lu et al. (2025). *Intelligent Go-Explore*. ICLR. [arXiv:2405.15143](#) — Go-Explore driven by foundation models

Reward design, hacking, and inverse RL:

- ▶ A. Y. Ng, D. Harada, S. Russell (1999). *Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping*. ICML.
- ▶ D. Amodei & J. Clark (2016). *Faulty Reward Functions in the Wild*. OpenAI blog. [openai.com/index/faulty-reward-functions](#)
- ▶ V. Krakovna et al. (2020). *Specification gaming: the flip side of AI ingenuity*. DeepMind blog. [deepmind.google](#)

- ▶ A. Y. Ng & S. Russell (2000). *Algorithms for Inverse Reinforcement Learning*. ICML.
- ▶ P. Abbeel & A. Y. Ng (2004). *Apprenticeship Learning via Inverse Reinforcement Learning*. ICML.
- ▶ B. D. Ziebart, A. Maas, J. A. Bagnell, A. K. Dey (2008). *Maximum Entropy Inverse Reinforcement Learning*. AAAI. [CMU PDF](#)
- ▶ C. Finn, S. Levine, P. Abbeel (2016). *Guided Cost Learning: Deep Inverse Optimal Control via Policy Optimization*. ICML. [arXiv:1603.00448](#)
- ▶ C. Finn, P. Christiano, P. Abbeel, S. Levine (2016). *A Connection between GANs, Inverse RL, and Energy-Based Models*. [arXiv:1611.03852](#)
- ▶ J. Ho & S. Ermon (2016). *Generative Adversarial Imitation Learning*. NeurIPS. [arXiv:1606.03476](#) — GAIL
- ▶ J. Fu, K. Luo, S. Levine (2018). *Learning Robust Rewards with Adversarial Inverse Reinforcement Learning*. ICLR. [arXiv:1710.11248](#) — AIRL

- ▶ P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, D. Amodei (2017). *Deep Reinforcement Learning from Human Preferences*. NeurIPS. [arXiv:1706.03741](#)
- ▶ *Inverse Reinforcement Learning Meets Large Language Models* (2025). [arXiv:2507.13158](#)
— reward modelling in RLHF as an IRL problem
- ▶ *Learning Reasoning Rewards from Expert Demonstrations with Inverse RL* (2025). [arXiv:2510.01857](#) — R-AIRL